INFO8006 Introduction to Artificial Intelligence

Exercise session 7: Making decisions and reinforcement learning

Recap

Making decisions

Markov decision processes. A *Markov decision process* (MDP) is a tuple $(\mathcal{S}, \mathcal{A}, P, R)$ such that

- $\mathcal{S}$ is a set of *states* s;
- $\mathcal{A}$ is a set of *actions* a;
- P is a (stationary) *transition model* such that $P(s' \mid s, a)$ denotes the probability of reaching state s' if action a is done in state s;
- R is a *reward function* that maps immediate (finite) reward values $R(s)$ obtained in states s,

and $0 < \gamma \leq 1$ is the *discount factor*.

Utilities and policies. The *discounted utility* of a reward sequence is $V([r_0, r_1, r_2, \ldots]) = r_0 + \gamma r_1 + \gamma^2 r_2 + \ldots$, which is additive for $\gamma = 1$. A *policy* $\pi : \mathcal{S} \to \mathcal{A}$ maps states to actions, and the expected utility obtained by executing π starting in s is

$$V^\pi(s) = \mathbb{E}\left[\sum_{t \geq 0} \gamma^t R(s_t) \mid s_0 = s, \pi\right].$$

The *optimal policy* π^* maximizes the expected utility, and the *value* of a state is $V(s) = V^{\pi^*}(s)$. Then, $\pi^*(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) V(s')$.

Bellman equation. The utility of a state is the immediate reward for that state, plus the expected discounted utility of the next state, assuming that the agent chooses the optimal action:

$$V(s) = R(s) + \gamma \max_a \sum_{s'} P(s' \mid s, a) V(s').$$

When the reward depends on the transition, $R(s, a, s')$, it moves under the sum: $V(s) = \max_a \sum_{s'} P(s' \mid s, a)[R(s, a, s') + \gamma V(s')]$.

Value iteration. Starting from initial estimates $V_0(s)$, *value iteration* repeats the *Bellman update*

$$V_{i+1}(s) = R(s) + \gamma \max_a \sum_{s'} P(s' \mid s, a) V_i(s')$$

until convergence. It converges to the unique solution of the Bellman equations whenever $\gamma < 1$, and also for $\gamma = 1$ when every policy reaches a terminal state.

Policy iteration. Policy iteration alternates *policy evaluation*, which calculates $V_i = V^{\pi_i}$ by solving the linear equations $V_i(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_i(s)) V_i(s')$, and *policy improvement*,

$$\pi_{i+1}(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) V_i(s').$$

Modified policy iteration replaces the exact evaluation by a few simplified Bellman updates $V_{i+1}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_i(s)) V_i(s')$.

Partially observable MDPs. When the state is not observed, the agent collects percepts e through a sensor model $P(e \mid s)$ and maintains a *belief state* b, a distribution over states. After doing a and perceiving e, it becomes

$$b' = \alpha \mathbf{P}(e \mid S') \sum_s \mathbf{P}(S' \mid s, a) b(s).$$

The optimal action depends only on b: a POMDP is an MDP over belief states, with the reward $\rho(b) = \sum_s b(s) R(s)$.

Reinforcement learning

Reinforcement learning. In *reinforcement learning*, the transition model $P(s' \mid s, a)$ and the reward function $R(s)$ are unknown: the agent learns from trials, whose trajectories are sequences of transitions (s, r, a, s'), each holding a state, its reward $r = R(s)$, the action done and the next state. In *passive RL*, the policy π is fixed and the agent learns the utilities $V^\pi(s)$; in *active RL*, it learns the optimal policy π^*. A *model-based* agent estimates $\hat{P}(s' \mid s, a)$ and $\hat{R}(s)$ from experience and solves the resulting empirical MDP. A *model-free* agent does not; for instance, *direct utility estimation* approximates $V^\pi(s)$ by the sample average of the observed rewards-to-go from s.

Temporal-difference learning. Temporal-difference (TD) *learning* updates $V^\pi(s)$ each time the agent experiences a transition $(s, r = R(s), a = \pi(s), s')$, to better agree with the Bellman equations for a fixed policy:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha(r + \gamma V^\pi(s') - V^\pi(s)),$$

where α is the *learning rate*.

Exploration. An agent that always acts greedily with respect to its current estimates may never discover better actions. An *ϵ-greedy* agent acts randomly with a small probability ϵ and follows its current policy otherwise. An *exploration function* $f(v, n)$, such as $f(v, n) = v + K/n$, instead makes the agent optimistic about the actions it has tried n times only, in $V^+_{i+1}(s) = R(s) + \gamma \max_a f\left(\sum_{s'} P(s' \mid s, a) V^+_i(s'), N(s, a)\right)$.

Q-Learning. The *state-action-value* $Q(s, a)$ is the expected utility starting out having taken action a from s and thereafter acting optimally, so that $V(s) = \max_a Q(s, a)$ and $\pi^*(s) = \arg\max_a Q(s, a)$. *Q-Learning* learns it in a model-free fashion, by updating $Q(s, a)$ each time the agent experiences a transition (s, r, a, s'):

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)).$$

It converges to the optimal Q-values, even when acting suboptimally, provided it explores enough. In *approximate Q-Learning*, $Q(s, a) = w_1 f_1(s, a) + \ldots + w_d f_d(s, a)$ is a linear model of features $f_k(s, a)$, and the update becomes $w_k \leftarrow w_k + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)) f_k(s, a)$.

Exercises

In session: exercises 1, 4.

Making decisions

Exercise 1 Micro-Blackjack (CS188, Spring 2014)
A card-playing program plays Micro-Blackjack. It repeatedly draws a card (with replacement), equally likely to be a 2, 3 or 4, and adds its value to its score, which starts at 0. At each step, if the score is lower than 6, the program can either "draw" (d) or "cash" (c). Otherwise, it can only cash. When it cashes, the game stops and its utility is equal to its score (up to 5) plus 1, or zero if the score is 6 or higher. Until it cashes and after it, it receives no reward. There is no discount ($\gamma = 1$).

(a) Formalize Micro-Blackjack as an MDP.

Solution. A state $s = (s_1, s_2)$ holds the score $s_1 \in \{0, 2, 3, 4, 5, 6+\}$ (all scores of 6 or more behave alike) and whether the program has cashed, $s_2 \in \{0, 1\}$. The states with $s_2 = 1$ are terminal. The available actions are

$$\mathcal{A}(s) = \begin{cases} \{d, c\} & \text{if } s_1 < 6 \text{ and } s_2 = 0, \\ \{c\} & \text{if } s_1 \geq 6 \text{ and } s_2 = 0, \\ \{\} & \text{otherwise,} \end{cases}$$

and the transition model and the reward function are

$$P(s' = (s'_1, s'_2) \mid s = (s_1, s_2), a) = \begin{cases} 1 & \text{if } a = c,\ s'_1 = s_1 \text{ and } s'_2 = 1, \\ \frac{1}{3} & \text{if } a = d,\ s'_1 - s_1 \in \{2, 3, 4\} \text{ and } s'_2 = 0, \\ 0 & \text{otherwise,} \end{cases}$$

$$R(s = (s_1, s_2)) = \begin{cases} s_1 + 1 & \text{if } s_1 < 6 \text{ and } s_2 = 1, \\ 0 & \text{otherwise.} \end{cases}$$

(b) Derive the optimal policy for this MDP.

Solution. The value $V(s)$ of a state, its maximum expected utility, satisfies the Bellman equation

$$V(s) = R(s) + \gamma \max_{a \in \mathcal{A}(s)} \sum_{s'} P(s' \mid s, a) V(s'),$$

and the optimal policy is $\pi^*(s) = \arg\max_{a \in \mathcal{A}(s)} \sum_{s'} P(s' \mid s, a) V(s')$. Since no action is available in the terminal states, their value is their reward, $V((s_1, 1)) = R((s_1, 1))$. We can apply either value iteration or policy iteration.

- In value iteration, we start from $V_0(s) = R(s)$ and repeat the Bellman update

$$V_{i+1}(s) = R(s) + \gamma \max_{a \in \mathcal{A}(s)} \sum_{s'} P(s' \mid s, a) V_i(s')$$

until convergence ($V_{i+1} = V_i$). Although $\gamma = 1$, it converges, since every game ends: every policy reaches a terminal state within four steps. For example, for the first state of the first iteration, we have

$$\begin{aligned} V_1((0,0)) &= R((0,0)) + \gamma \max \left\{ \underbrace{V_0((0,1))}_{a=c},\ \underbrace{\tfrac{1}{3} V_0((2,0)) + \tfrac{1}{3} V_0((3,0)) + \tfrac{1}{3} V_0((4,0))}_{a=d} \right\} \\ &= 0 + \max\{1,\ 0\} = 1. \end{aligned}$$

s_2	0						1					
s_1	0	2	3	4	5	6+	0	2	3	4	5	6+
$V_0(s)$	0	0	0	0	0	0	1	3	4	5	6	0
$V_1(s)$	1	3	4	5	6	0	1	3	4	5	6	0
$V_2(s)$	4	$\frac{11}{3}$	4	5	6	0	1	3	4	5	6	0
$V_3(s)$	$\frac{38}{9}$	$\frac{11}{3}$	4	5	6	0	1	3	4	5	6	0
$V_4(s)$	$\frac{38}{9}$	$\frac{11}{3}$	4	5	6	0	1	3	4	5	6	0

Each iteration takes $\mathcal{O}(|\mathcal{S}|^2|\mathcal{A}|)$ operations. The optimal policy follows from the values:

s_2	0						1					
s_1	0	2	3	4	5	6+	0	2	3	4	5	6+
$\pi^*(s)$	d	d	c	c	c	c				–		

- In policy iteration, we start from a policy π_0, evaluate it by solving the linear equations

$$V^{\pi_i}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_i(s)) V^{\pi_i}(s'),$$

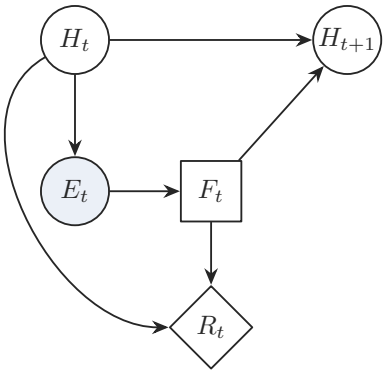
and improve it with $\pi_{i+1}(s) = \arg\max_{a \in \mathcal{A}(s)} \sum_{s'} P(s' \mid s, a) V^{\pi_i}(s')$, until the policy no longer changes. Here, the equations are triangular, since the score only increases, and are solved from the highest score down.
With $\pi_0(s) = c$ everywhere, $V^{\pi_0}((s_1, 0)) = s_1 + 1$ for $s_1 < 6$ and $V^{\pi_0}((6+, 0)) = 0$. Improving it, drawing from $(0, 0)$ is worth $\frac{1}{3}(3 + 4 + 5) = 4 > 1$, from $(2, 0)$ it is worth $\frac{1}{3}(5 + 6 + 0) = \frac{11}{3} > 3$, and from $(3, 0)$ only $\frac{1}{3}(6 + 0 + 0) = 2 < 4$; drawing from $(4, 0)$ or $(5, 0)$ always busts. Hence $\pi_1 = (d, d, c, c, c, c)$ for $s_1 = 0, 2, 3, 4, 5, 6+$.
Evaluating π_1 from the highest score down gives $V^{\pi_1}((5, 0)) = 6$, $V^{\pi_1}((4, 0)) = 5$, $V^{\pi_1}((3, 0)) = 4$, $V^{\pi_1}((2, 0)) = \frac{1}{3}(5 + 6 + 0) = \frac{11}{3}$ and $V^{\pi_1}((0, 0)) = \frac{1}{3}(\frac{11}{3} + 4 + 5) = \frac{38}{9}$. Improving π_1 changes no action ($\frac{38}{9} > 1$, $\frac{11}{3} > 3$, $2 < 4$), so that $\pi_2 = \pi_1 = \pi^*$, the policy found by value iteration.
For large problems, solving the linear equations exactly costs $\mathcal{O}(|\mathcal{S}|^3)$ operations; *modified* policy iteration then replaces the exact evaluation by a few simplified Bellman updates $V_{j+1}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_i(s)) V_j(s')$.

Exercise 2 Overheating machine

A maintenance robot looks after a machine that may overheat. It cannot measure the temperature directly: every hour, it only checks whether the machine's alarm is on, and decides whether to cool the machine for the next hour. The alarm is on with probability 0.8 when the machine is overheating, and with probability 0.1 when it is not. Cooling costs 5 (energy), whereas leaving an overheating machine without cooling for an hour costs 10 (wear). Between two checks, a machine that is not overheating starts overheating with probability 0.2, an overheating machine that is not cooled keeps overheating, and a machine that has been cooled is no longer overheating at the next check. When the robot starts, the machine is overheating with probability 0.5.

(a) Draw the dynamic decision network of this problem, as well as its conditional probability tables and its reward function.

Solution. Let $H_t \in \{0, 1\}$ indicate whether the machine is overheating at time t, $E_t \in \{0, 1\}$ whether the alarm is on, and $F_t \in \{0, 1\}$ whether the robot cools the machine. The reward is $R_t \in \{0, -5, -10\}$. The alarm is observed, and the decision depends on it.



H_t	$P(E_t = 1 \mid H_t)$
0	0.1
1	0.8

H_t	F_t	$P(H_{t+1} = 1 \mid H_t, F_t)$	$R_t(H_t, F_t)$
0	0	0.2	0
1	0	1	−10
0	1	0	−5
1	1	0	−5

For completeness we also have to define the prior $P(H_0 = 1) = 0.5$. Note that R_t is a deterministic function of H_t and F_t.

(b) Give the sequence of belief states if the robot observes the alarm and acts as follows: $(e_t, f_t) = (0, 0)$, $(1, 1)$, $(0, 0)$, $(0, 0)$ for $t = 0, 1, 2, 3$.

Solution. The environment is only partially observable (POMDP): the robot does not know the current state $s = H_t$, but perceives $e = E_t$ through the sensor model $P(e \mid s)$ and acts $a = F_t$. If b is the current belief state and the robot does a and then perceives e', the next belief state is

$$b'(s') = \alpha P(e' \mid s') \sum_s P(s' \mid s, a) b(s).$$

Here, this gives

$$b_{t+1}(h_{t+1}) = \alpha P(e_{t+1} \mid h_{t+1}) \sum_{h_t} P(h_{t+1} \mid h_t, f_t) b_t(h_t),$$

with the initial belief

$$b_0 = P(H_0 \mid e_0) = \alpha P(e_0 \mid H_0) P(H_0).$$

With $e_{0:3} = f_{0:3} = (0, 1, 0, 0)$ and beliefs written as $(b(0), b(1))$, we have

$$b_0 = \alpha_0 \begin{pmatrix} 0.9 & 0 \\ 0 & 0.2 \end{pmatrix} \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} \approx \begin{pmatrix} 0.8182 \\ 0.1818 \end{pmatrix}$$

$$b_1 = \alpha_1 \begin{pmatrix} 0.1 & 0 \\ 0 & 0.8 \end{pmatrix} \begin{pmatrix} 0.8 & 0 \\ 0.2 & 1 \end{pmatrix} b_0 \approx \begin{pmatrix} 0.1915 \\ 0.8085 \end{pmatrix}$$

$$b_2 = \alpha_2 \begin{pmatrix} 0.9 & 0 \\ 0 & 0.2 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} b_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$b_3 = \alpha_3 \begin{pmatrix} 0.9 & 0 \\ 0 & 0.2 \end{pmatrix} \begin{pmatrix} 0.8 & 0 \\ 0.2 & 1 \end{pmatrix} b_2 \approx \begin{pmatrix} 0.9474 \\ 0.0526 \end{pmatrix}.$$

(c) From the sequence of belief states, determine the sequence of expected rewards.

Solution. The expected reward of a belief state b is $\rho(b) = \sum_s b(s)R(s)$. Here, the reward also depends on the action:

$$\rho_t = \sum_{h_t} R_t(h_t, f_t) b_t(h_t).$$

Then,

$$\rho_0 = \begin{pmatrix} 0 & -10 \end{pmatrix} b_0 \approx -1.818$$

$$\rho_1 = \begin{pmatrix} -5 & -5 \end{pmatrix} b_1 = -5$$

$$\rho_2 = \begin{pmatrix} 0 & -10 \end{pmatrix} b_2 = 0$$

$$\rho_3 = \begin{pmatrix} 0 & -10 \end{pmatrix} b_3 \approx -0.526.$$

Exercise 3 Pursuit evasion (CS188, Spring 2014)

Pacman is trapped in the following 2 by 2 maze with a hungry ghost. When it is his turn to move, Pacman must move one step horizontally or vertically to a neighboring square. When it is the ghost's turn, he must also move one step horizontally or vertically. The ghost and Pacman alternate moves. After every move (by either the ghost or Pacman), if Pacman and the ghost occupy the same square, Pacman is eaten and receive utility -100. Otherwise, he receives utility of 1. The ghost attempts to minimize the utility that Pacman receives. Pacman makes the first move. The game is not guaranteed to terminate.



For example, with a discount factor of $\gamma = 1$, if Pacman moves right, ghost moves down, then Pacman moves left, Pacman earns a reward of 1 initially ($R(s_0)$) and after the two first moves and -100 after the last move, for a total utility of -97.

(a) Assume a discount factor $\gamma = 0.5$, where the discount factor is applied once every time either Pacman or the ghost moves. What is the Minimax value of the complete (infinite) game?

Solution. If we assume the game starts from the displayed state s_0, Pacman has always an action to get out of reach of the ghost. Hence, if Pacman is optimal, which is true by definition in Minimax, he will never get eaten. Therefore, we can compute the initial state value

$$V(s_0) = \sum_{t=0}^{\infty} \gamma^t \underbrace{R(s_t)}_{1} = \lim_{t \to \infty} \frac{1 - \gamma^{t+1}}{1 - \gamma} = \frac{1}{1 - \gamma} = 2,$$

without actually building the (infinite) Minimax tree.

(b) Why are the value/policy iteration algorithms superior to Minimax for solving this game?

Solution. If $\gamma < 1$, we know that the value/policy iteration algorithms converge toward a fixed point, even if the game does not terminate. Conversely, Minimax is not guaranteed to terminate due to its recursivity.

This game is similar to an MDP because rewards are earned at every timestep. However, it is also an adversarial game involving decisions by two agents. Let s be the state (*e.g.* the positions of Pacman and the ghost and who is playing) and let $A(s)$ be the set of actions available to the player in state s. Let $s' = T(s, a)$ denote the successor of a state s resulting from the action a. Finally, let $R(s)$ denote the utility received after moving to state s.

(c) Write down an expression (analogous to Bellman equation) for $V(s)$, the value of the current state if it is Pacman's turn.

Solution.

$$V(s) = R(s) + \gamma \max_{a \in A(s)} \left[R(T(s,a)) + \gamma \min_{a' \in A(T(s,a))} V(T(T(s,a), a')) \right]$$

Reinforcement learning

Exercise 4 Q-learning

An agent is in an unknown environment where there are three states $\{A, B, C\}$ and two actions $\{0, 1\}$. We are given the following tuples (s, a, r, s'), generated by taking actions in the environment.

s	a	r	s'
A	0	$+2$	A
C	1	-2	A
B	1	$+1$	B
A	0	-1	B
B	1	-2	C
C	0	$+4$	B
B	0	$+1$	A

Assuming a discount factor $\gamma = 0.5$ and a learning rate $\alpha = 0.75$,

(a) Apply the Q-learning algorithm to obtain state-action-value $Q(s, a)$ estimates. Estimates are initialized to 0.

Solution. As seen in the previous exercise, given trial trajectories of some policy π, it is possible to estimate its expected utility V^π even without knowing the transition model $P(s' \mid s, a)$ or the reward function $R(s)$ of the environment. However, estimating V^π only evaluates the quality of π, but does not describe how to improve it.

By definition, the optimal policy π^* is the one that maximizes the expect utility of the state s, *i.e.*

$$\pi^*(s) = \arg\max_a \sum_{s'} P(s' \mid s, a) V(s')$$

where $V(s) = \max_\pi V^\pi(s)$ is the state-value of s. Unfortunately, even if we knew V, we could not find the optimal actions without knowing the transition model. However, if we knew the state-action-value $Q(s, a)$ of taking action a in state s, we would be able to select the optimal action

$$\pi^*(s) = \arg\max_a Q(s, a),$$

where

$$\begin{aligned} Q(s,a) &= R(s) + \gamma \sum_{s'} P(s' \mid s,a) V(s') \\ &= R(s) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a'} Q(s',a') \end{aligned}$$

since $V(s) = \max_a Q(s,a)$. Fortunately, state-action-values $Q(s,a)$ can be learned in a model-free fashion using the Q-learning algorithm. Similarly to temporal-difference learning, in Q-learning, we perform stochastic gradient descent updates

$$\begin{aligned} \hat{Q}(s,a) &\leftarrow \hat{Q}(s,a) - \alpha\Big(\hat{Q}(s,a) - r - \gamma \max_{a'} \hat{Q}(s',a')\Big) \\ &\leftarrow (1-\alpha)\hat{Q}(s,a) + \alpha\Big(r + \gamma \max_{a'} \hat{Q}(s',a')\Big) \end{aligned}$$

from observed tuples (s,r,a,s').
In our case, the sequence of updates would be

$$\begin{aligned} \hat{Q}(A,0) &\leftarrow (1-\alpha)\hat{Q}(A,0) + \alpha\Big(2 + \gamma \max\{\hat{Q}(A,0), \hat{Q}(A,1)\}\Big) \\ &= \frac{1}{4}0 + \frac{3}{4}(2 + \frac{1}{2}0) = \frac{+3}{2} \\ \hat{Q}(C,1) &\leftarrow (1-\alpha)\hat{Q}(C,1) + \alpha\Big(-2 + \gamma \max\{\hat{Q}(A,0), \hat{Q}(A,1)\}\Big) \\ &= \frac{1}{4}0 + \frac{3}{4}(-2 + \frac{1}{2}\frac{3}{2}) = \frac{-15}{16} \\ \hat{Q}(B,1) &\leftarrow (1-\alpha)\hat{Q}(B,1) + \alpha\Big(1 + \gamma \max\{\hat{Q}(B,0), \hat{Q}(B,1)\}\Big) \\ &= \frac{1}{4}0 + \frac{3}{4}(1 + \frac{1}{2}0) = \frac{+3}{4} \\ \hat{Q}(A,0) &\leftarrow (1-\alpha)\hat{Q}(A,0) + \alpha\Big(-1 + \gamma \max\{\hat{Q}(B,0), \hat{Q}(B,1)\}\Big) \\ &= \frac{1}{4}\frac{3}{2} + \frac{3}{4}(-1 + \frac{1}{2}\frac{3}{4}) = \frac{-3}{32} \\ \hat{Q}(B,1) &\leftarrow \dots \end{aligned}$$

It should be noted that, as for TD learning, we are not limited to perform only one update for each tuple. Reusing tuples several times (in other orders) is a good way to reach convergence faster while generating less trials.

(b) We now switch to a feature-based estimator $\hat{Q}(s,a) = w_0 + w_1 f_1(s,a)$, with $f_1(s,a) = 2a - 1$. Starting from weights $w_0 = w_1 = 0$, update the weights according to the approximate Q-learning algorithm.

Solution. We would like our estimates $\hat{Q}(s,a)$ to satisfy the Bellman equation, *i.e.* to minimize

$$L = \mathop{\mathbb{E}}_{s'|s,a}\left[\left(R(s) + \gamma \max_{a'} \hat{Q}(s',a') - \hat{Q}(s,a)\right)^2\right].$$

To reach this objective we follow the opposite of its gradient

$$\nabla_w L = 2 \mathop{\mathbb{E}}_{s'|s,a}\left[\left(\hat{Q}(s,a) - R(s) - \gamma \max_{a'} \hat{Q}(s',a')\right)\nabla_w \hat{Q}(s,a)\right]$$

with respect to the weights[1] w of $\hat{Q}(s,a)$. Once again, due to the expectation, we are forced to use a stochastic approximation of the gradient from observed tuples (s,a,r,s') and to update the weights as

$$w \leftarrow w - \alpha\Big(\hat{Q}(s,a) - r - \gamma \max_{a'} \hat{Q}(s',a')\Big)\nabla_w \hat{Q}(s,a),$$

where α is the learning rate. In our case,

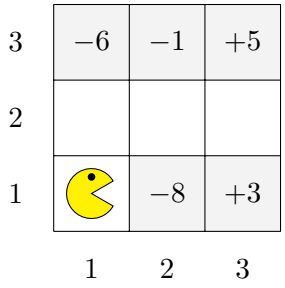
$$\nabla_w \hat{Q}(s,a) = \begin{pmatrix} \partial_{w_0} \\ \partial_{w_1} \end{pmatrix} (w_0 + w_1 f_1(s,a)) = \begin{pmatrix} 1 \\ f_1(s,a) \end{pmatrix} = \begin{pmatrix} 1 \\ 2a-1 \end{pmatrix}$$

and the two first weight updates would be

$$\begin{aligned} w &\leftarrow w - \alpha\Big(\hat{Q}(A,0) - 2 - \gamma \max\{\hat{Q}(A,0), \hat{Q}(A,1)\}\Big)\nabla_w \hat{Q}(A,0) \\ &\leftarrow \begin{pmatrix} 0 \\ 0 \end{pmatrix} - \frac{3}{4}\Big(-2 - \frac{1}{2}\max\{0,0\}\Big)\begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} \frac{+3}{2} \\ \frac{-3}{2} \end{pmatrix} \\ w &\leftarrow w - \alpha\Big(\hat{Q}(C,1) + 2 - \gamma \max\{\hat{Q}(A,0), \hat{Q}(A,1)\}\Big)\nabla_w \hat{Q}(C,1) \\ &\leftarrow \begin{pmatrix} \frac{+3}{2} \\ \frac{-3}{2} \end{pmatrix} - \frac{3}{4}\Big(2 - \frac{1}{2}\max\{0,3\}\Big)\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{+9}{8} \\ \frac{-15}{8} \end{pmatrix}. \end{aligned}$$

Exercise 5 Passive reinforcement learning

Consider the grid-world below and an agent who is trying to learn the optimal policy. The agent starts from the bottom-left corner and can take the actions `north` (N), `south` (S), `west` (W) and `east` (E). Rewards are only awarded for reaching the terminal (shaded) states.



You observe the following trials, whose trajectories are sequences of tuples $(s_t^i, r_t^i, a_t^i, s_{t+1}^i)$.

t	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5
0	$(1,1), 0, N, (1,2)$	$(1,1), 0, N, (1,2)$	$(1,1), 0, N, (1,2)$	$(1,1), 0, N, (1,2)$	$(1,1), 0, N, (1,2)$
1	$(1,2), 0, E, (2,2)$	$(1,2), 0, E, (2,2)$	$(1,2), 0, E, (2,2)$	$(1,2), 0, E, (2,2)$	$(1,2), 0, E, (2,2)$
2	$(2,2), 0, N, (2,3)$	$(2,2), 0, E, (3,2)$	$(2,2), 0, S, (2,1)$	$(2,2), 0, E, (3,2)$	$(2,2), 0, E, (3,2)$
3	$(2,3), -1, \varnothing, \varnothing$	$(3,2), 0, N, (3,3)$	$(2,1), -8, \varnothing, \varnothing$	$(3,2), 0, W, (2,2)$	$(3,2), 0, S, (3,1)$
4		$(3,3), +5, \varnothing, \varnothing$		$(2,2), 0, N, (2,3)$	$(3,1), +3, \varnothing, \varnothing$
5				$(2,3), -1, \varnothing, \varnothing$	

Assuming a discount factor $\gamma = 1$,

[1] It should be noted that the target $R(s) + \gamma \max_{a'} \hat{Q}(s', a')$ should not affected by the gradient operation as it is considered constant in the objective. In practice, this is not true as modifying the weights in a parametric function $\hat{Q}$ does so for all state-action pairs (s, a) at once. This generally makes the approximate Q-learning algorithm very unstable.

(a) Perform direct utility estimation of the expected utilities $V^\pi(s)$, given the four first trials.

Solution. In this setting, the transition model $P(s' \mid s, a)$ and the reward function $R(s)$ are unknown. We wish to learn the expected utility V^π of the policy π without modeling the environment, *i.e.* without building approximates $\hat{P}(s' \mid s, a)$ and $\hat{R}(s)$. The principle of direct utility estimation is to approximate $V^\pi(s)$ by the average utility $\hat{V}(s)$ of the state s within all trial trajectories, *i.e.*

$$\hat{V}(s) = \frac{1}{N(s)} \sum_{(i,j)\in I(s)} \sum_{t=0}^{\infty} \gamma^t r^i_{j+t} \approx V^\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]\Bigg|_{s_0=s}$$

where $I(s) = \{(i,j) : s^i_j = s\}$ indexes the occurrences of s and $N(s) = |I(s)|$ is the number of occurrences of s. For examples, in our case and excluding the fifth trial,

$$\begin{aligned}
I((1,1)) &= \{(1,0),(2,0),(3,0),(4,0)\} \\
\hat{V}((1,1)) &\approx \frac{1}{4}\left(-1\,\gamma^3 + 5\,\gamma^4 - 8\,\gamma^3 - 1\,\gamma^5\right) = \frac{-5}{4} \\
I((2,2)) &= \{(1,2),(2,2),(3,2),(4,2),(4,4)\} \\
\hat{V}((2,2)) &\approx \frac{1}{5}\left(-1\,\gamma^1 + 5\,\gamma^2 - 8\,\gamma^1 - 1\,\gamma^3 - 1\,\gamma^1\right) = \frac{-6}{5} \\
I((2,3)) &= \{(1,3),(4,5)\} \\
\hat{V}((2,3)) &\approx \frac{1}{2}\left(-1\,\gamma^0 - 1\,\gamma^0\right) = -1.
\end{aligned}$$

Importantly, since the agent hasn't reached the states $(1,3)$ and $(3,3)$ yet, it is not possible to estimate their expected utility. Instead, we assume a default value of 0.

3	0	−1	+5
2	$\frac{-5}{4}$	$\frac{-6}{5}$	+2
1	$\frac{-5}{4}$	−8	0
	1	2	3

(b) Update the estimated expected utilities with respect to the fifth trial using temporal-difference learning. Assume a learning rate $\alpha = 0.5$.

Solution. Direct utility estimation misses the fact that the expected utilities for different states are not independent, since they obey the Bellman equation for a fixed policy

$$V^\pi(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s)) V^\pi(s').$$

Then, we would like our estimates $\hat{V}(s)$ to verify the Bellman equation, *i.e.* to minimize

$$L = \mathop{\mathbb{E}}_{s'|s}\left[\left(R(s) + \gamma \hat{V}(s') - \hat{V}(s)\right)^2\right].$$

One way to reach this objective is to follow the opposite of its gradient

$$\nabla L = 2 \mathop{\mathbb{E}}_{s'|s}\left[\hat{V}(s) - R(s) - \gamma \hat{V}(s')\right]$$

with respect to $\hat{V}(s)$, *i.e.* to perform *gradient descent*. Unfortunately, due to the expectation, we don't have access to the true gradient. Instead, we use a *stochastic* approximation of

the gradient from observed[2] tuples (s, r, a, s') and update $\hat{V}(s)$ as

$$\begin{aligned}\hat{V}(s) &\leftarrow \hat{V}(s) - \alpha\Big(\hat{V}(s) - r - \gamma\hat{V}(s')\Big)\\ &\leftarrow (1-\alpha)\hat{V}(s) + \alpha\Big(r + \gamma\hat{V}(s')\Big)\end{aligned}$$

where α is the learning rate. For a fixed learning rate, the average of $\hat{V}(s)$ converge towards (oscillates around) the true expected utility $V^\pi(s)$. For a (slowly) decreasing learning rate, $\hat{V}(s)$ itself converges to $V^\pi(s)$. For historical reasons, this algorithm is called temporal-difference (TD) learning.

Following the trajectory of the fifth trial, we perform the updates

$$\begin{aligned}
V^\pi((1,1)) &\leftarrow (1-\alpha)V^\pi((1,1)) + \alpha(0 + \gamma V^\pi((1,2))) = \frac{1}{2}\frac{-5}{4} + \frac{1}{2}\frac{-5}{4} = \frac{-5}{4}\\
V^\pi((1,2)) &\leftarrow (1-\alpha)V^\pi((1,2)) + \alpha(0 + \gamma V^\pi((2,2))) = \frac{1}{2}\frac{-5}{4} + \frac{1}{2}\frac{-6}{5} = \frac{-49}{40}\\
V^\pi((2,2)) &\leftarrow (1-\alpha)V^\pi((2,2)) + \alpha(0 + \gamma V^\pi((3,2))) = \frac{1}{2}\frac{-6}{5} + \frac{1}{2}2 = \frac{+4}{10}\\
V^\pi((3,2)) &\leftarrow (1-\alpha)V^\pi((3,2)) + \alpha(0 + \gamma V^\pi((3,1))) = \frac{1}{2}2 + \frac{1}{2}0 = +1\\
V^\pi((3,1)) &\leftarrow (1-\alpha)V^\pi((3,1)) + \alpha(3 + 0) = \frac{1}{2}0 + \frac{1}{2}3 = \frac{+3}{2}.
\end{aligned}$$

3	0	-1	$+5$
2	$\frac{-49}{40}$	$\frac{+4}{10}$	$+1$
1	$\frac{-5}{4}$	-8	$\frac{+3}{2}$
	1	2	3

[2]We are not limited to perform only one update for each tuple (s, r, a, s'). Indeed, as we assume the policy to be stationary, past trials are as likely to happen again as more recent ones. Therefore, reusing tuples several times (in other orders) is a good way to improve estimates while generating less trials.

Exercise 6 Football

ULiège's football team is playing against UCL's team next week. With a lot of losses this season, Liège needs to improve their attack strategy to win the game and increase their popularity. Luckily, the team captain follows INFO8006 and knows how to model the attack as a Markov Decision Process. The captain considers four states `close` (`C`), `away` (`A`), `fail` (`F`), and `goal` (`G`), and two actions `pass` (`P`) and `shoot` (`S`). Although the transition probabilities are unsure, the possible transitions (s, a, s') are known. To each transition is associated an increase/decrease of the team's popularity.

s	a	s'	$R(s,a,s')$
C	P	C	$+1$
C	P	A	-1
C	P	F	-2
C	S	C	$+3$
C	S	F	-5
C	S	G	$+10$
A	P	C	$+2$
A	P	A	0
A	P	F	-3
A	S	C	$+3$
A	S	F	-10
A	S	G	$+20$

The current strategy of the team is to always shoot. Last match, they had several attack opportunities, resulting in the following actions.

s	C	C	C	C	A	A	A	A
a	S	S	S	S	S	S	S	S
s'	G	C	G	F	F	C	F	F

Assuming a discount factor $\gamma = 0.75$ and a learning rate $\alpha = 0.25$,

(a) Build an estimator of the transition model $P(s' \mid s, a)$ and, from it, determine the expected utility V^π of the team's current policy π.

Solution. Given the previous table, we can estimate the transition probabilities for $s \in \{\texttt{C}, \texttt{A}\}$ and $a = \texttt{S}$.

s	a	$P(s' \mid s, a)$			
		C	A	F	G
C	S	0.25	0	0.25	0.5
A	S	0.25	0	0.75	0

We can now determine the expected utility V^π of the team's current policy $\pi(s) = \texttt{S}$ using the following Bellman operator

$$V^\pi_{i+1}(s) = \sum_{s'} P(s' \mid s, \pi(s))\big[R(s, \pi(s), s') + \gamma V^\pi_i(s')\big],$$

with $V^\pi_0(s) = 0$ for all state s. Because the states `F`and `G`are terminal, their expected utility remains constant and we only have to update `C`and `A`. For the first iteration,

$$\begin{aligned} V^\pi_1(\texttt{C}) &= \sum_{s'} P(s' \mid \texttt{C}, \texttt{S})\big[R(\texttt{C}, \texttt{S}, s') + \gamma V^\pi_0(s')\big] \\ &= 0.25(3 + \gamma\, 0) + 0.25(-5 + \gamma\, 0) + 0.5(10 + \gamma\, 0) = 4.5 \end{aligned}$$

and

$$V_1^\pi(\mathtt{A}) = \sum_{s'} P(s' \mid \mathtt{A}, \mathtt{S})\big[R(\mathtt{A}, \mathtt{S}, s') + \gamma V_0^\pi(s')\big]$$
$$= 0.25(3 + \gamma\, 0) + 0.75(-10 + \gamma\, 0) = -6.75.$$

We repeat the same procedure until $V_{i+1}^\pi(s) \approx V_i^\pi(s)$ for all state s.

s	C	A	F	G
$V_0^\pi(s)$	0	0	0	0
$V_1^\pi(s)$	4.5	-6.75	0	0
$V_2^\pi(s)$	5.3437	-5.9063	0	0
$V_3^\pi(s)$	5.5019	-5.7480	0	0
$V_4^\pi(s)$	5.5316	-5.7183	0	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$V_8^\pi(s)$	5.5385	-5.7115	0	0

The captain found the tapes of the previous season where they had much more success. Together with the team, the captain selects the following instructive actions.

s	C	C	A	A	C	A	A	C
a	S	S	S	P	P	P	S	P
s'	G	C	F	C	C	C	F	F

(b) Given the selected tuples, apply the Q-learning algorithm to obtain state-action-value $Q(s, a)$ estimates. Estimates are initialized to 0.

Solution. In Q-learning, the state-action-value $Q(s, a)$ estimates are updated following

$$Q(s,a) \leftarrow Q(s,a) - \alpha\Big(Q(s,a) - r - \gamma \max_{a'} Q(s', a')\Big)$$
$$\leftarrow (1-\alpha)Q(s,a) + \alpha\Big(r + \gamma \max_{a'} Q(s', a')\Big)$$

for observed tuples (s, r, a, s'). In our case, we have a list a 16 $(8 + 8)$ tuples (s, a, s') and we have access to the reward function $R(s, a, s')$. Using the tuples in arbitrary order, we have

$$Q(\mathtt{C},\mathtt{S}) \leftarrow (1-\alpha)Q(\mathtt{C},\mathtt{S}) + \alpha\Big(R(\mathtt{C},\mathtt{S},\mathtt{G}) + \gamma \max_{a'} Q(\mathtt{G}, a')\Big)$$
$$\leftarrow 0.75 \times 0 + 0.25(10 + 0) = 2.5$$
$$Q(\mathtt{C},\mathtt{S}) \leftarrow (1-\alpha)Q(\mathtt{C},\mathtt{S}) + \alpha\Big(R(\mathtt{C},\mathtt{S},\mathtt{C}) + \gamma \max_{a'} Q(\mathtt{C}, a')\Big)$$
$$\leftarrow 0.75 \times 2.5 + 0.25(3 + 0.75 \max\{2.5, 0\}) = 3.094$$
$$Q(\mathtt{A},\mathtt{S}) \leftarrow (1-\alpha)Q(\mathtt{A},\mathtt{S}) + \alpha\Big(R(\mathtt{A},\mathtt{S},\mathtt{F}) + \gamma \max_{a'} Q(\mathtt{F}, a')\Big)$$
$$\leftarrow 0.75 \times 0 + 0.25(-10 + 0) = -2.5$$
$$Q(\mathtt{A},\mathtt{P}) \leftarrow (1-\alpha)Q(\mathtt{A},\mathtt{P}) + \alpha\Big(R(\mathtt{A},\mathtt{P},\mathtt{C}) + \gamma \max_{a'} Q(\mathtt{C}, a')\Big)$$
$$\leftarrow 0.75 \times 0 + 0.25(2 + 0.75 \max\{3.094, 0\}) = 4.320$$
$$Q(\mathtt{C},\mathtt{P}) \leftarrow \ldots$$

(c) Determine the optimal policy according to the state-action-value estimates.

Solution.

$$\pi(s) = \begin{cases} \texttt{S} & \text{if } s = \texttt{C} \\ \texttt{P} & \text{if } s = \texttt{A} \end{cases}$$

Supplementary exercises

Exercise 7 Buying the textbook (AIMA ch. 16)

Consider a student, Sam, who has the choice to buy or not buy a textbook for a course. We'll model this as a decision problem with one Boolean decision node, B, indicating whether the agent chooses to buy the book, and two Boolean chance nodes, M, indicating whether the student has mastered the material of the course, and F, indicating whether the student fails the course. There is also a utility node, U. Sam has an additive utility function: 0 for not buying the book and $-100\,\$$ for buying it; and 0 for failing the course and $2000\,\$$ for passing it. Sam's conditional probability estimates are as follows:

B	M	$P(M \mid B)$	$P(F = 1 \mid B, M)$
0	0	0.3	0.7
1	0	0.1	0.5
0	1	0.7	0.2
1	1	0.9	0.1

(a) Draw the decision network for this problem.

(b) Compute the expected utility of buying the book and of not buying it.

(c) What should Sam do?

Exercise 8 Contraction of the Bellman operator (AIMA ch. 17)

The slides of the course states that the Bellman operator

$$(BV)(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s' \mid s, a) V(s')$$

is a contraction.

(a) Show that, for any two functions $f : \mathbb{R} \mapsto \mathbb{R}$ and $g : \mathbb{R} \mapsto \mathbb{R}$,

$$|\max_x f(x) - \max_x g(x)| \leq \max_x |f(x) - g(x)|.$$

(b) Apply the previous result to prove that the Bellman operator is a contraction of factor γ, that is

$$\|(BV) - (BV')\|_\infty \leq \gamma \|V - V'\|_\infty.$$

Exercise 9 Value iteration (AIMA ch. 17)

Consider the 3×3 world shown in Figure 1. The transition model is the same as in the 4×3 2: $80\,\%$ of the time the agent goes in the direction it selects; the rest of the time it moves at right angles to the intended direction.

Implement value iteration for this world for each value of r below. Use discounted rewards with a discount factor $\gamma = 0.99$. Show the policy obtained in each case. Explain intuitively why the value of r leads to each policy.

The infinity norm of a function $f : \mathbb{R} \mapsto \mathbb{R}$ is defined as

$$\|f\|_\infty = \sup_x |f(x)|.$$

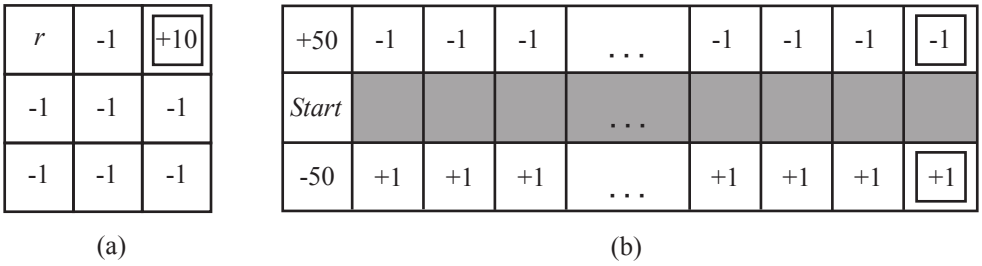


Figure 1. (a) 3×3 world for Exercise 9. The reward for each state is indicated. The upper right square is a terminal state. (b) 101×3 world for Exercise 10. The start state has reward 0.

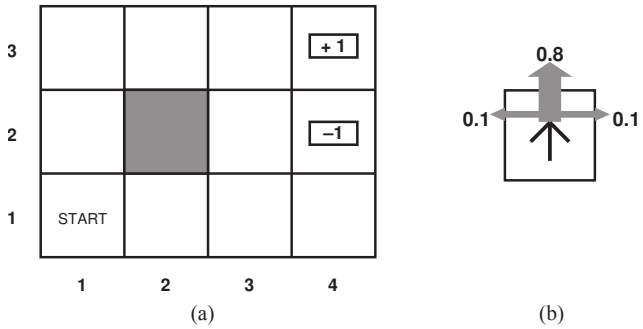


Figure 2. (a) A simple 4×3 environment that presents the agent with a sequential decision problem. The two terminal states have reward $+1$ and -1, respectively, and all other states have a reward of -0.04. (b) Illustration of the transition model of the environment: the "intended" outcome occurs with probability 0.8, but with probability 0.2 the agent moves at right angles to the intended direction. A collision with a wall results in no movement.

(a) $r = -100$

(b) $r = -3$

(c) $r = 0$

(d) $r = +3$

Exercise 10 Discount factor (AIMA ch. 17)

Consider the 101×3 world shown in Figure 1. In the start state the agent has a choice of two deterministic actions, `up` or `down`, but in the other states the agent has one deterministic action, `right`. Assuming a discounted reward function, compute the utility of each action as a function of γ. For what values of the discount factor γ should the agent choose `up` and for which `down`?

Exercise 11 Policy iteration (AIMA ch. 17)

Consider an undiscounted MDP having three states, $(1, 2, 3)$, with rewards -1, -2, 0, respectively. State 3 is a terminal state. In states 1 and 2 there are two possible actions: a and b. The transition model is as follows:

- In state 1, action a moves the agent to state 2 with probability 0.8 and makes the agent stay put with probability 0.2.
- In state 2, action a moves the agent to state 1 with probability 0.8 and makes the agent stay put with probability 0.2.

This simple example actually reflects many real-world situations in which one must weigh the value of an immediate action versus the potential long-term consequences, such as choosing to dump pollutants into a lake.

- In either state 1 or state 2, action b moves the agent to state 3 with probability 0.1 and makes the agent stay put with probability 0.9.

Answer the following questions:

(a) What can be determined *qualitatively* about the optimal policy in states 1 and 2?

(b) Apply policy iteration, showing each step in full, to determine the optimal policy and the values of states 1 and 2. Assume that the initial policy has action b in both states.

(c) What happens to policy iteration if the initial policy has action a in both states? Does discounting help? Does the optimal policy depend on the discount factor?

Exercise 12 Dutch auction (AIMA ch. 18)

A Dutch auction is similar to an English auction, but rather than starting the bidding at a low price and increasing, the seller starts at a high price and gradually lowers the price until some buyer is willing to accept that price. If multiple bidders accept the price, one is arbitrarily chosen as the winner. More formally, the seller begins with a price p and gradually lowers p by increments of d until at least one buyer accepts the price.

Assuming all bidders act rationally, is it true that for arbitrarily small d, a Dutch auction will always result in the bidder with the highest value for the item obtaining the item? If so, show mathematically why. If not, explain how it may be possible for the bidder with highest value for the item not to obtain it.

Exercise 13 Hockey overtime (AIMA ch. 18)

Teams in the National Hockey League historically received 2 points for winning a game and 0 for losing. If the game is tied, an overtime period is played; if nobody wins in overtime, the game is a tie and each team gets 1 point. But league officials felt that teams were playing too conservatively in overtime (to avoid a loss), and it would be more exciting if overtime produced a winner. So in 1999 the officials experimented in mechanism design: the rules were changed, giving a team that loses in overtime 1 point, not 0. It is still 2 points for a win and 1 for a tie.

(a) Was hockey a zero-sum game before the rule change? After?

(b) Suppose that at a certain time t in a game, the home team has probability p of winning in regulation time, probability $0.78 - p$ of losing, and probability 0.22 of going into overtime, where they have probability q of winning, $0.9 - q$ of losing, and 0.1 of tying. Give equations for the expected value for the home and visiting teams.

(c) Imagine that it were legal and ethical for the two teams to enter into a pact where they agree that they will skate to a tie in regulation time, and then both try in earnest to win in overtime. Under what conditions, in terms of p and q, would it be rational for both teams to agree to this pact?

(d) Some experts report that since the rule change, the percentage of games with a winner in overtime went up 18.2 %, as desired, but the percentage of overtime games also went up 3.6 %. What does that suggest about possible collusion or conservative play after the rule change?

Quiz

1. Which of the following is true? In Markov Decision Processes, ...
   - ◯ the closer the discount factor γ to 0, the higher the utility of future rewards.
   - ◯ the closer the discount factor γ to 0, the longer Value Iteration may take to converge.
   - ◯ the closer the discount factor γ to 1, the greedier the optimal agent.
   - ● the closer the discount factor γ to 1, the longer Value Iteration may take to converge.
2. Q-Learning ...
   - ◯ is a model-based reinforcement learning algorithm.
   - ◯ is an on-policy reinforcement learning algorithm.
   - ◯ converges to an optimal policy, but only when acting optimally.
   - ● requires a random exploration strategy to converge to an optimal policy.
3. Assume that we run ϵ-greedy Q-learning until convergence. What is the optimal policy π^* we obtain for an arbitrary state s?
   - ◯ $\pi^*(s) = \arg\max_s V(s)$
   - ◯ $\pi^*(s) = \begin{cases} \arg\max_a Q(s,a) & \text{with probability } 1-\epsilon \\ \text{random action} & \text{with probability } \epsilon \end{cases}$
   - ● $\pi^*(s) = \arg\max_a Q(s,a)$
   - ◯ $\pi^*(s) = \arg\max_s Q(s,a)$
4. Let us consider a robot wandering around at the Montefiore Institute. Which of the following is true?
   - ● The robot and its environment can be modeled as a partially observable MDP.
   - ◯ The Kalman filter can be used for determining accurately its past trajectory in the building.
   - ◯ A convolutional neural network would be too large to fit in the robot's memory.
   - ◯ Value iteration (with an admissible heuristic) can be used for decoding the speech of its visitors.
5. The Bellman equations $Q(s,a) = R(s) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a'} Q(s',a')$ form a system of n non-linear equations with as many unknowns. Which of the following is true?
   - ◯ n is the size of the action space $\mathcal{A}$.
   - ◯ n is the size of the state space $\mathcal{S}$.
   - ● n is the size of the state-action space $\mathcal{S} \times \mathcal{A}$.
   - ◯ n is the sum of the sizes of the action and state spaces.
6. In DQN (Mnih et al, 2015), a reinforcement learning agent is trained to ...
   - ◯ to classify images.
   - ◯ to control a robot in a simulated environment.
   - ● to play Atari video games.
   - ◯ to play the game of Go.
7. In DQN (Mnih et al, 2015), the Q-table is approximated with ...
   - ◯ a hash table.
   - ◯ a transposition table.
   - ◯ a linear regression model.
   - ● a convolutional neural network.